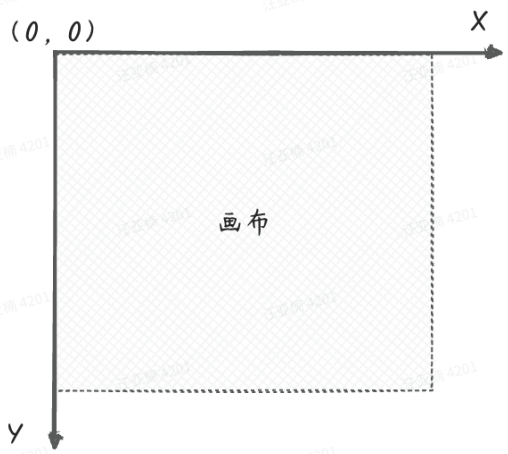
变换（transform）

如果你不懂画布中的变换（transform），很多效果都会变得难以实现，如果你想不使用变换绘制一张旋转45度的图像，要么你就提前将图像旋转45度，要么就是通过像素级处理，所以在我看来，画布变换并不是画布进阶的知识，而是是必须要了解的入门级的知识。

一、基本使用

1）原点和坐标系

画布提供了部分API给开发者快速实现画布变换的操作，在认识它们之前我们需要先认识画布的**原点**和**坐标系**。



如上图，画布原点通常被定义为画布的左上角，其坐标为(0, 0)。原点的意义在于让所有绘制的图形元素都相对于该点进行定位。

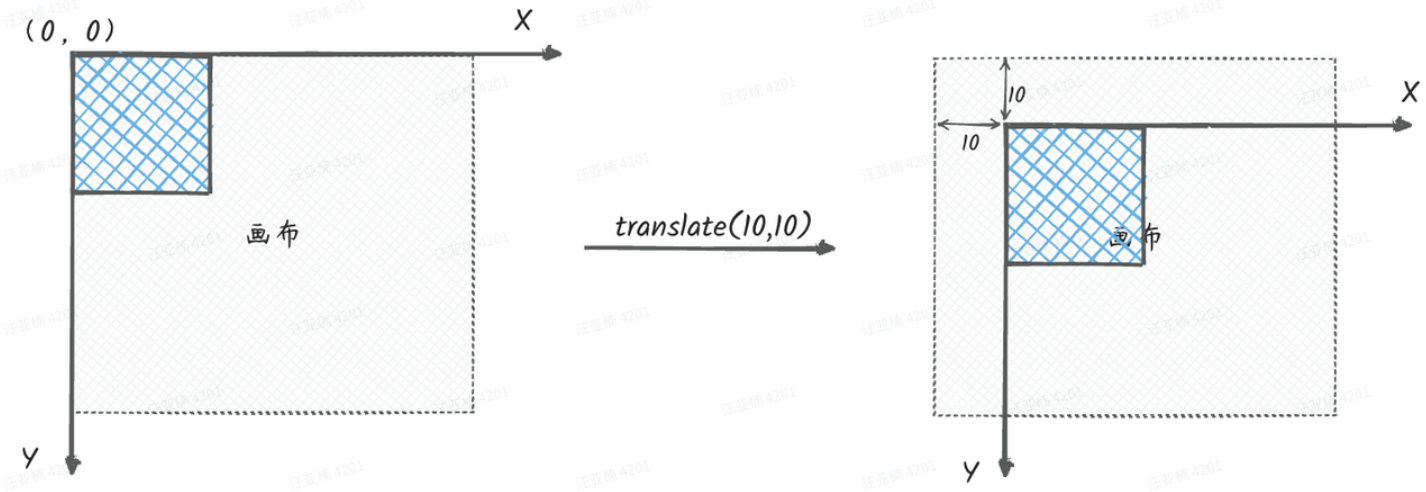
而画布的坐标系是一个二维的笛卡尔坐标系，但与数学中的坐标系不太一样，在计算机图形学中，由于屏幕的物理特性，y轴通常是从上到下延伸，值从上到下增加。

当你对画布进行变换（如旋转、缩放、倾斜或平移）时，这些变换都是相当于画布的原点和坐标系进行的。

2）原生API

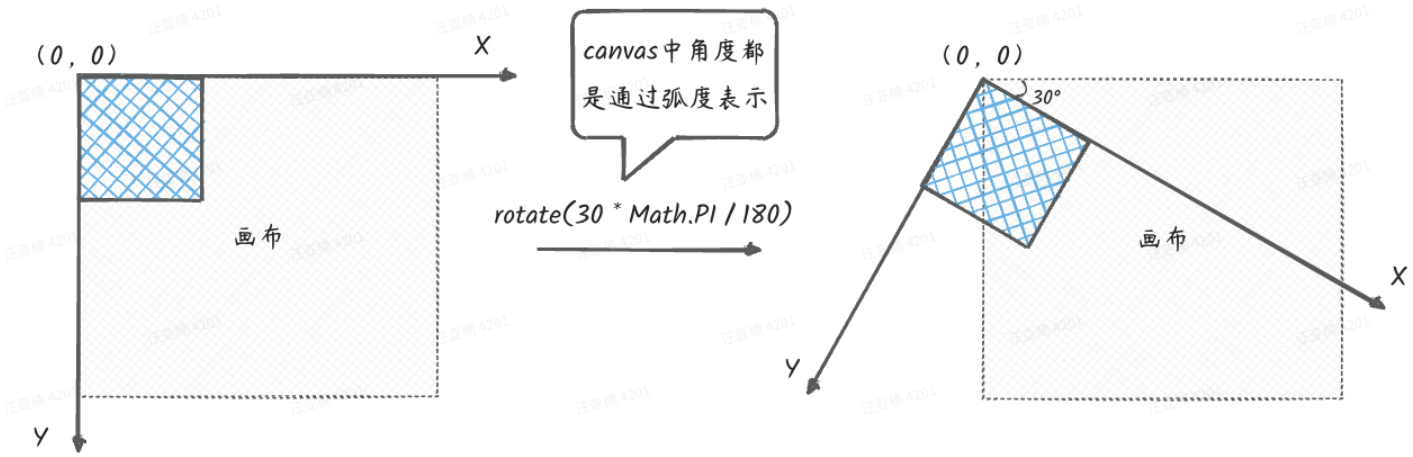
① `translate(x, y)` **移动**

将画布原点移动到x，y的位置，如下图所示：



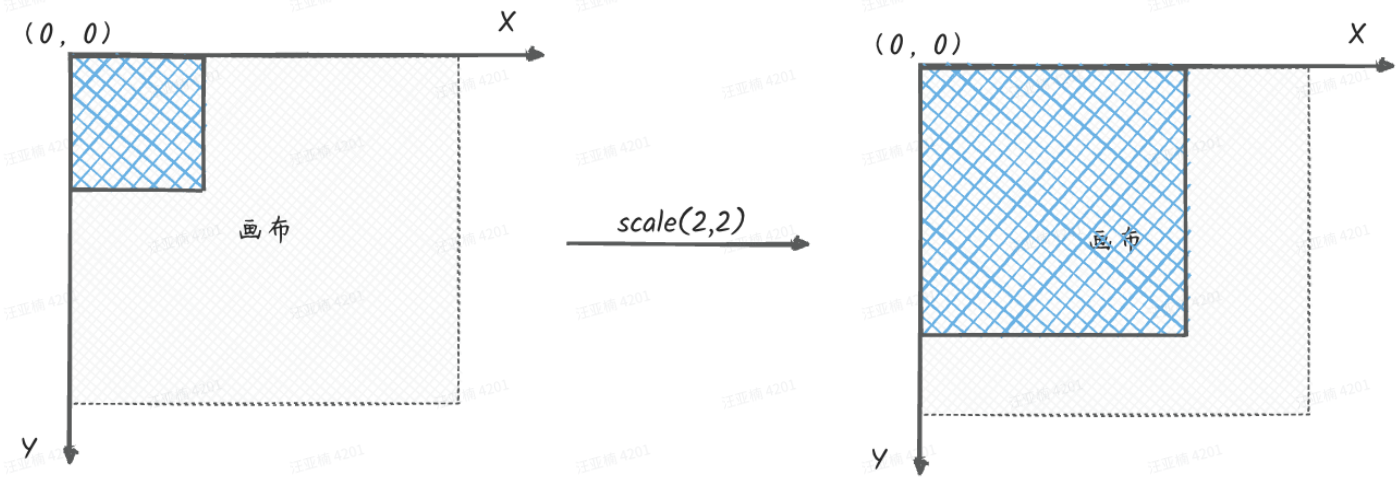
② `rotate(angle)` 旋转

画布原点不变，但坐标系会进行旋转，如下图所示：



③ `scale(sx, sy)` 缩放

不改变画布的原点和坐标系，但是在后续的绘制中，一个单位长度不再对应一个像素，而是对应sx个像素（对于x轴）或sy个像素（对于y轴），如下图所示：

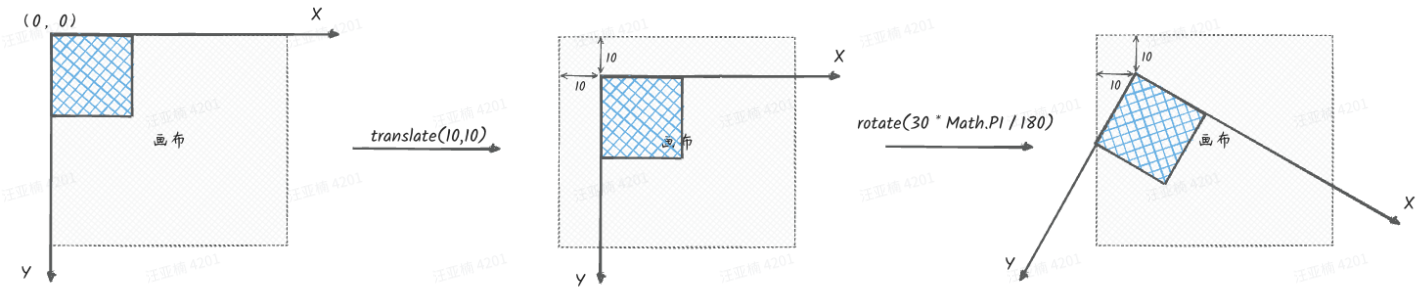


当缩放值变为负数时，可巧妙用于实现镜像效果，但要注意其镜像效果相对于当前原点：

https://codepen.io/huanjinliu/embed/RwqEmKP?editors=1010

3）变换的效果是累积的

画布中的变化的效果都是累积的，当你调用 `translate(10, 10)` 后再调用 `rotate(20 * Math.PI / 180)` ，画布的原点会先移动到 `(10, 10)` 的位置，然后再旋转30°。如下图所示：



如果你不知道这个特点，你很容易会因为错误的变换而导致自己的绘制坐标系超出画布可视区域。比如最常见的就是在循环中调用 `translate` ，因为变换效果是累积的，原点会因不断偏移而超出画布区域导致绘图消失。

如果要设置临时的变换，可以使用我们之前学习的 `save & restore` 状态栈来实现状态的保存和恢复：

```
let ctx = canvas.getContext('2d');
// 临时偏移原点（10, 10）
ctx.save();
ctx.translate(10, 10);
ctx.restore();
// 临时旋转坐标系30°
ctx.save();
ctx.rotate(30 * Math.PI / 180);
ctx.restore();
```

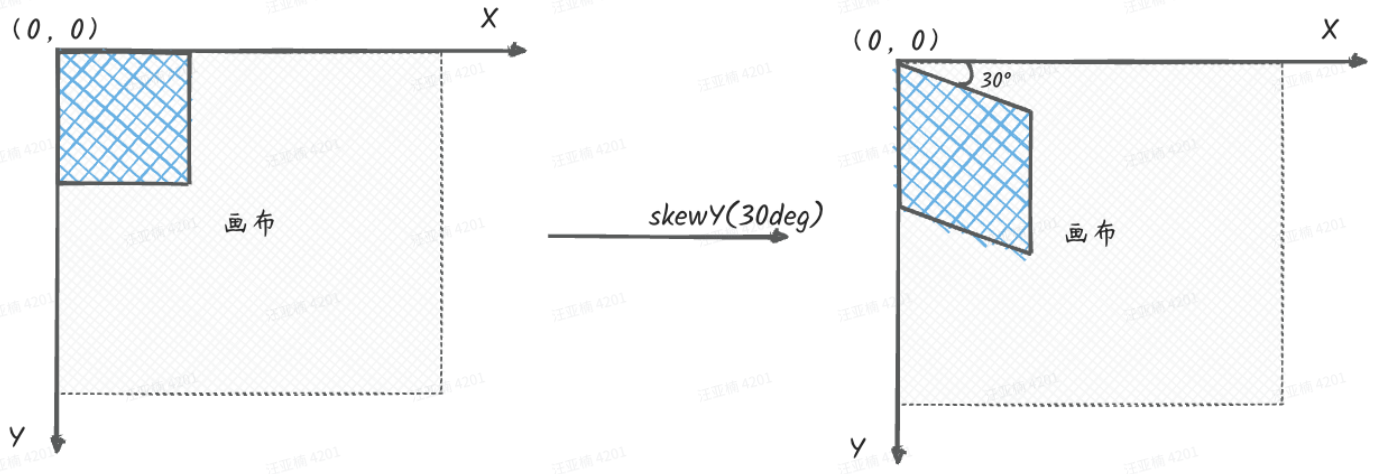
二、示例演示：实现车轮绘制及旋转

https://codepen.io/huanjinliu/embed/wvQQLgQ?editors=1010

对比通过三角函数计算去绘制，变换的实现更为简单，且某些情况下动画实现也更加方便。

三、 `transform(a, b, c, d, e, f)`

我们已经学习了画布的三个核心API：偏移、旋转和缩放，这些知识对于我们后续的开发已经足够使用了，但是还是有一种情况没办法通过它们实现，那就是倾斜变形，如下图所示：



在某些场景下，你可能会想通过 `canvas2D` 实现 `3D` 的效果，这时候图像的变形是免不了，但是普通的 `translate`、 `ratote`、 `scale` 是不能实现我们想要的效果的。这时就需要用到画布中比较难理解的 `transform(a, b, c, d, e, f)` API了。

而要理解这个API就先需要了解在图形学中的矩阵运算了： 矩阵运算